

JAVA

- Only object oriented language
- Made by James Goslin (initially called as OAK)
- It's first compiled into byte code & then interpreted into machine code



→ package com.company; → Package

public class Main { → class

public static void main(String[] args) { → Func

System.out.println("Hello World");

}

}

- Package is used to group related classes

→ Basic structure of java program

Function → To do a task

Class → creates objects to use functions

→ Naming conventions

1. Pascal naming conventions

- used to name classes
- eg → Main, My Scanner, CodeWithHarry etc
- i.e. First letter of every word is capital

(2) Camel case conventions

- used for functions & variables, first lower case then upper case. eg → main, myScanner etc.

→ Variable

AVAT

- stores a value that can be changed

(200) eg. `int number = 8;` `number` is variable, `8` is value.
 ↓
 data type

→ How to name a variable

- can't start with a digit
- var name is case sensitive
- can't be a keywords like `public` / `static`
- space not allowed first name X

`first - name` ✓ ← `firstName` ✓
`first name` ✗ ← `first Name` ✗

→ Primitive data types

- JAVA (is statically) typed i.e. variables must be declared before use

- There are 8 primitive data types

1. `byte` → 1 byte i.e. 8 bits
2. `short` → 2 bytes
3. `int` → 4 bytes
4. `float` → 4 bytes
5. `long` → 8 bytes
6. `double` → 8 bytes
7. `char` → 2 bytes
8. `boolean` → depends on JVM True / False

Primitive data types

Integral

`byte` `short` `long` `int`

(L)

Floats

`float`
(F, f)

`double`

`char`
(`'`, `"`)

`Boolean`
(`T`, `F`)

• (→ Literals: "100", "10.1f", "10.1", "true", "100")

• constant value which can be assigned to a variable is called literal

101 → integer

10.1f → float

10.1 → double

"100" → String

true → boolean

"100" → String

100 → int

10.1f → float

10.1 → double

"100" → String

true → boolean

"100" → String

100 → int

10.1f → float

10.1 → double

"100" → String

true → boolean

"100" → String

100 → int

10.1f → float

10.1 → double

"100" → String

true → boolean

"100" → String

100 → int

10.1f → float

10.1 → double

"100" → String

• (→ Taking inputs)

• JAVA has a Scanner class to take data

from user

import java.util.Scanner;

Scanner sc = new Scanner(System.in);

Scanner class object

int a = sc.nextInt();

Reads (takes) value from user

Program for sum of 3 nos.

import java.util.Scanner;

public class Main {

public static void main(String[] args) {

Scanner sc = new Scanner(System.in);

System.out.print("Enter num 1:");

int num1 = sc.nextInt();

System.out.print("Enter num 2:");

int num2 = sc.nextInt();

System.out.print("Enter num 3:");

int num3 = sc.nextInt();

int sum = (num1 + num2 + num3);

System.out.print("The sum is:" + sum);

Q Program to calculate CGPA using marks of three subjects

```
import java.util.Scanner;
public class Main {
    public static void main (String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Marks in sub 1:");
        int sub1 = sc.nextInt();
        System.out.print("Marks in sub 2:");
        int sub2 = sc.nextInt();
        System.out.print("Marks in sub 3:");
        int sub3 = sc.nextInt();
        float cgpa = ((sub1 + sub2 + sub3) / 30);
        System.out.print("CGPA = " + cgpa);
    }
}
```

Q Take name; and print a greeting msg

```
import java.util.Scanner;
public class Main {
    public static void main (String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter your name:");
        String name = sc.next();
        System.out.print("Hello " + name + " Have a good day!");
    }
}
```

Q Convert kilometers to miles

```
import java.util.Scanner;
public class Main {
    public static void main (String [] args) {
        Scanner sc = new Scanner (System.in);
        System.out.print ("Enter distance in km:");
        float km = sc.nextFloat();
        float miles = (km * 0.62f);
        System.out.print ("Distance in miles is " + miles);
    }
}
```

Q Program to check if entered number is integer or not

```
import java.util.Scanner;
public class Main {
    public static void main (String [] args) {
        Scanner sc = new Scanner (System.in);
        System.out.print ("Enter a number");
        System.out.println (sc.hasNextInt());
    }
}
```

↓
gives true if int is entered
otherwise gives false

→ Scanning syntax for different data types

int int var = sc.nextInt();

float float var = sc.nextFloat();

double double var = sc.nextDouble();

Single word string string var = sc.next();

Full line string string var = sc.nextLine();

Boolean boolean var = sc.nextBoolean();

→ operators

- used to perform operations on variables and values

1. Arithmetic $+$, $-$, $*$, $/$, $\%$, $++$, $--$

2. Assignment $=$, $+=$, $-=$

3. Comparison $==$, $!=$, $>$, $<$, $>=$, $<=$

4. Logical $\&\&$, $\|\|$, $!$

5. Bitwise $\&$, $\|$, $\sim$, $\ll$, $\gg$

→ Precedence of operators

() $>$ $*$ / $>$ $+$ / $-$

If precedence of two operators is same then we use associativity to determine the direction

$*$ / $\rightarrow$ Left to Right

$+$ $-$ $\rightarrow$ Left to Right

$++$, $--$ $\rightarrow$ Right to Left

Left to right

eg: $a = 6 * 5 - 34 / 2$

$= 30 - 17$

$= 13$

Left to right

eg: $b = 6 / 5 - 34 * 2$

$= 12 - 68$

$= -56$

→ byte + short → int
 short + int → int
 long + float → float
 int + float → float
 char + int → int
 char + short → int
 long + double → double
 float + double → double

→ Increment & Decrement operators

- $a++$, $++a$ → increment operators
- $a--$, $--a$ → decrement operators
- Operate on all data types except boolean
- $a++$: First use the variable & then increment it
- $++a$: First increment & then use
- If $\text{char } a = 'B'$ then $a++$ means $a = 'C'$ now

Q float $a = 7/4 * 9/2$
 same precedence, associativity, left to right
 $a = 1.75 * 9/2$
 $= 15.75/2$
 $= 7.875$

Q Java program to encrypt a grade by adding 8 to it, decrypt it & show the correct grade

```
import java.util.Scanner;
public class Main {
    public static void main (String [] Args)
    {
```

```

Scanner sc = new Scanner(System.in);
System.out.print("Enter the grade you want to encrypt");
char grade = sc.next().charAt(0);
char encrypted = grade + 8;
char decrypted = (char)(encrypted - 8);
System.out.print("Encrypted grade: " + encrypted);
System.out.print("Correct grade: " + decrypted);

```

Use comparison operators to find out whether a given number is greater than the user entered or not.

```

import java.util.Scanner;

public class Main {
    public static void main (String [] args) {
        Scanner sc = new Scanner (System.in);
        int given_number = 72;
        System.out.print("Enter a number");
        int num = sc.nextInt();
        boolean isGreater = (num > given_number);
        System.out.print("Is entered number greater than given number? " + isGreater);
    }
}

```

Q Write this expression in JAVA program

```

import java.util.Scanner;
public class Main {
    public static void main (String [] args) {
        Scanner sc = new Scanner (System.in);
        System.out.print ("Enter value of v");
        float v = sc.nextFloat();
        System.out.print ("Enter value of u");
        float u = sc.nextFloat();
        System.out.print ("Enter value of a");
        float a = sc.nextFloat();
        System.out.print ("Enter value of s");
        float s = sc.nextFloat();
        float x = (v*v - u*u) / (2 * a * s);
        System.out.print ("Value of solved expression is", + x);
    }
}

```

Q Find value of a

```
int x = 7;
```

```
int a = 7 * 49/7 + 35/7;
```

$$a = 7 * 49/7 + 35/7$$

$$= 343/7 + 35/7$$

$$= 49 + 35/7$$

$$= 49 + 5$$

$$= 54$$

→ Strings

- A string is a sequence of characters
- Strings are immutable and can't be changed
- A string can be used like a data type but is actually a class

String declaration :-

```
String name;
```

```
name = new String("Harry");
```

String name = "Harry"; → is datatype

→ ways to print in JAVA

1. `System.out.print()` → no new line in end
2. `System.out.println()` → new line in end
3. `System.out.printf()`
4. `System.out.format()`

```
System.out.printf("%c", ch);
```

- ↳ `%d` for int
- ↳ `%c` for char
- ↳ `%f` for float
- ↳ `%s` for string

→ String Methods

- Used to operate on java strings. They can be used to :-
- 1. find length of string
- 2. convert to lower case
- 3. convert to upper case
- and many more

(for eg $\rightarrow$ `String name = "HARRY"`)

0 1 2 3 4

1. `name.length()`

- It will return the length of string name.
- (5 in this case)
- i.e. index + 1

2. `name.toLowerCase()`

- Would return a new string by converting all characters to "lowercase" of name.
- "harry" in this case

3. `name.toUpperCase()`

- Returns new string with all upper case characters

4. `name.trim()`

- Returns a new string after removing all leading & trailing spaces
- eg $\rightarrow$ if `name = "Hello World"`
`name.trim()` gives 'HelloWorld'

5. `name.substring(int start)`

- Returns a substring from starting index given upto end.
- eg if `name = "Harry"`
`name.substring(3)` returns 'ry'

6. `name.substring (int start, int end)`

- Returns substring from start to end index
- Includes start index but excludes end index

eg → `name = "Harry"`
`name.substring (1, 4);` gives 'arr'

7. `name.replace (char 'initial', char 'final')`

- Returns a new string after replacing the initial character with final char

eg → `name = "Harry"`
`name.replace ('r', 'p');` returns "Happy"

8. `name.startsWith (" ")`

- Returns true if name starts with given string

eg → `name = "Harish"`

`name.startsWith ("Ha");` returns True

9. `name.endsWith (" ")`

- Returns true if name ends with the given string

10. `name.charAt (int)`

Returns the character present at the given int index in name.

eg → `name = "Malik";`

`name.charAt (2);`
 returns 'l'

11. `name.index Of ("string")`

Returns the index of given string.

• eg → `name = "Harry"`

`name.index Of ("ar")`

would return 1 which is the first occurrence of ar in string "Harry", -1

otherwise

`name.index Of ("a") = -1`

12. `name.index Of ("s", 3)`

Returns the index of a given string starting from the given int. index (3 in this case)

• eg → `name = "Harry"`

`name.index Of ("s", 3)` searches for "s" in string 'ry' but returns -1 as it's not present

13. `name.last Index Of ("r")`

Returns the last index of given string

• eg → Harry me bar bar arha hai
at 2 and 3 index, to use function use index ki value bataiye. `name.last Index Of ("r")` will return 3 as bar arha hai

14. `name.last Index Of ("r", 2)`

Returns the last index of given string before the index 2

15. `name.equals ("Harry")`

Returns true if the given string is exactly equal to "Harry"

• It's case sensitive

16. `name.equalsIgnoreCase("Harry")`
- Returns true if two strings are equal ignoring the case of characters.

→ "escape" sequence characters

- Sequence of characters after backslash '\'

escape sequence characters consists of more than one character but represents one character when used within the strings.

for eg → `\n`, `\t`, `\'`, `\\` etc

$\downarrow$ new line $\downarrow$ single space $\downarrow$ semicolon $\downarrow$ Back slash

Q. JAVA program to convert a string into lower case

```

import java.util.Scanner;
public class Main {
    public static void main (String [] args) {
        Scanner sc = new Scanner (System.in);
        System.out.print ("enter a string");
        String random = sc.nextLine();
        String converted = random.toLowerCase();
        System.out.print ("converted string is: " + converted);
    }
}
  
```

Q Program to replace spaces with underscores

```

import java.util.Scanner;
public class Main {
    public static void main (String [] args) {
        Scanner sc = new Scanner (System.in);
        System.out.print ("Express your
        feelings in one line");
        String feeling = sc.nextLine();
        String converted = feeling.replace(" ", "_");

        System.out.print ("The converted string
        is : " + converted);
    }
}

```

Q JAVA program to fill in a letter template which looks like

letter = "Dear <|name|>, Thanks a lot"

Replace <|name|> with a string

```

import java.util.Scanner;
public class Main {
    public static void main (String [] args) {
        String message = "Dear <|name|>, Thanks a lot";
        Scanner sc = new Scanner (System.in);
        System.out.print ("Enter your name:");
        String name = sc.nextLine();
        message = message.replace (<|name|>, name);
        System.out.printf ("%s", message);
    }
}

```

Q JAVA program to detect double & triple spaces in a string.

```

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String myString = "This string contains double & triple spaces.";
        System.out.println("Index of ' ' is: " + myString.indexOf(" "));
        System.out.println("Index of ' ' is: " + myString.indexOf(" "));
    }
}

```

```

        System.out.println("Index of ' ' is: " + myString.indexOf(" "));
        System.out.println("Index of ' ' is: " + myString.indexOf(" "));
    }
}

```

Q JAVA program to find the number of vowels in a string.

```

import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String myString = "This string contains double & triple spaces.";
        int vowels = 0;
        for (int i = 0; i < myString.length(); i++) {
            if (myString.charAt(i) == 'a' || myString.charAt(i) == 'e' || myString.charAt(i) == 'i' || myString.charAt(i) == 'o' || myString.charAt(i) == 'u') {
                vowels++;
            }
        }
        System.out.println("Number of vowels: " + vowels);
    }
}

```

```

        System.out.println("Number of vowels: " + vowels);
    }
}

```

```

        System.out.println("Number of vowels: " + vowels);
    }
}

```

```

        System.out.println("Number of vowels: " + vowels);
    }
}

```

→ Conditional statements

1. If else statement

if (cond. to be checked) {
statement if cond. is true;

else {
statement if cond is false;

→ Relational operators in JAVA

- Used to evaluate conditions inside statements
- eg → $==$, $>$, $<$, $<=$, $>=$, $!=$
- $=$ → for assigning some value
- $==$ → to check equality

→ Logical operators

1. AND (& &)

True if both are true

$$Y \& \& Y = Y$$

$$Y \& \& N = N$$

$$N \& \& Y = N$$

$$N \& \& N = N$$

2. OR (||)

True if any one of these is true

$$Y || Y = Y$$

$$Y || N = Y$$

$$N || Y = Y$$

$$N || N = N$$

3. NOT (!)

$$!Y = N \quad \& \quad !N = Y$$

→ else if clause

```
if ( ) {
```

```
    }
```

```
else if ( ) {
```

```
    }
```

```
else {
```

```
    }
```

→ switch case control instruction

- used to make a choice between no of alternatives for a variable

switch (var) { → var can be int,

case C1: char or string

```
// code;
```

```
break;
```

```
case C2:
```

```
// code;
```

```
break;
```

```
,
```

```
,
```

```
,
```

```
default:
```

```
// code;
```

```
}
```

```
X = X || Y
```

```
X = X || Y
```

```
X = X || Y
```

```
X = X || Y
```

Q Program to find whether a student is pass or fail if it requires atleast 30% in each subject & total 40%. Assume 3 subjects and take marks from user.

```
import java.util.Scanner;
public class Main {
    public static void main {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter marks of sub 1:");
        int sub1 = sc.nextInt();
        System.out.print("sub 2:");
        int sub2 = sc.nextInt();
        System.out.print("sub 3:");
        int sub3 = sc.nextInt();
        float percentage = (sub1 + sub2 + sub3) / 3.0f;
        if (sub1 >= 30 && sub2 >= 30 && sub3 >= 30
            && percentage >= 40) {
            System.out.print("Pass");
        }
        else {
            System.out.print("Fail");
        }
    }
}
```

→ Loops

1. while loop → checks condition → executes

```
while (boolean condition) {
    // statement
}
```

• executes as long as condition is true.

2. do while → executes → checks condition

guaranteed to execute at least once.

```
do {
    // code
} while (condition);
```

Example: `do { cout << "Hello\n"; } while (true);`

3. For loop

```
for (initialisation; condition; update) {
    // code
}
```

Example: `for (int i = 1; i <= 10; i++) { cout << i << " "; }`

• executes code specific no. of times

Example: `for (int i = 1; i <= 10; i++) { cout << i << " "; }`

→ Break Statement

After break statement compiler exits the loop whether condition satisfies or not.

Example: `for (int i = 1; i <= 10; i++) { if (i == 5) break; cout << i << " "; }`

→ Continue Statement

continue is used to immediately move to next iteration of the loop, skipping everything below continue for that iteration.

Q Print * * * *
 * * *
 * *

```
import java.util.Scanner;
public class Main {
    public static void main (String [] args) {
        Scanner sc = new Scanner(System.in);
        for (int i = 0; i < 4; i++) {
            for (int j = i; j < 4; j++) {
                System.out.print("*");
            }
        }
    }
}
```

```
System.out.print("\n");
```

```
}
```

```
}
```

Q sum of first 10 even no's
 sout ("enter value of n");
 Scanner sc = new Scanner(System.in);
 int n = sc.nextInt();

```
int sum = 0;
for (int i = 0; i <= 2n; i++) {
    if (i % 2 == 0) {
        sum += i;
    }
}
```

```
sout ("sum : " + sum);
```

```
}
```

```
}
```

Q Program to write table of a number
 sout ("enter n");

int n = sc.nextInt();

for (int i=0; i<=10; i++) {

sout (n+"*"+i+"=" + n*i);

}

Q For reversed table

for (int i=10; i>=0; i--) {

sout (n+"*"+i+"=" + n*i);

}

Q Factorial

sout ("enter n");

int n = sc.nextInt();

long factorial = 1;

for (i=1; i<=n; i++) {

factorial *= i;

}

sout ("factorial: " + factorial);

Instead of int, use long!

(++i, --i, i++, i--)

(0 == 5 * i)

i = i + 1

(mul + " : mul")

→ Arrays

- collection of similar data types

```
int [] marks; // arr = [] arr[i] true
```

```
marks = new int [5]; // arr. true
```

```
(";" OR (i+j) + "marks") true
```

```
int [] marks = new int [5]; // arr
```

```
(";" OR (i+j) + "marks") true
```

```
int [] marks = {100, 200}; // arr
```

```
(";" OR (i+j) + "marks") true
```

- To find length of array (arr) name.length

→ Displaying an array

```
for (int i=0; i < marks.length; i++) {
    sout(marks[i]);
}
```

```
}
```

```
OR for (int element : arr) {
```

For each loop

```
sout element;
```

```
}
```

→ 2D array

```
int [][] flats = new int [2] [3];
```

- Q create array of 5 floats & find its sum

```
float [] arr = new float [5]
```

```
float sum = 0;
```

```
for (int i=0; i < arr.length; i++) {
    sout ("enter the element " + (i+1) + ":");
```

```
arr[i] = sc.nextFloat();
```

```
(";" OR (i+j) + "marks") true
```

```
for (int i=0; i < arr.length; i++) {
    sum += arr[i];
```

```
}
```

```
sout ("sum is : " + sum);
```

```
}
```

Q Program to find if a given integer is present in an array or not.

```

int[] arr = new int[5];
for (int i = 0; i < arr.length; i++) {
    cout << "enter element " << (i+1) << ": ";
    arr[i] = sc.nextInt();
}
cout << "enter the no. to check";
int n = sc.nextInt();
boolean isPresent = false;
for (int i = 0; i < arr.length; i++) {
    if (arr[i] == n) {
        isPresent = true;
        break;
    }
}
if (isPresent) {
    cout << n << " is present";
} else {
    cout << n << " isn't present";
}

```

Q Program to find avg marks from an array.

```

int[] arr = new int[5];
for (int i = 0; i < arr.length; i++) {
    cout << "enter element " << (i+1) << ": ";
    arr[i] = sc.nextInt();
}
int sum = 0;
for (int i = 0; i < arr.length; i++) {
    sum += arr[i];
}

```

```
float average = sum / arr.length;
    sout ("average is : " + average);
}
```

Q Program to add two 2×3 matrices

```
int[][] matrix1 = new int[2][3];
sout ("Enter elements of the matrix");
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++) {
        sout ("element [" + i + "][" + j + "]: ");
        matrix1[i][j] = sc.nextInt();
    }
}
sout ("First matrix");
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++) {
        sout (matrix1[i][j] + " ");
    }
}
sout ();
```

```
int[][] array = new int[2][3];
sout ("Enter elements of matrix");
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++) {
        sout ("element [" + i + "][" + j + "]: ");
        array[i][j] = sc.nextInt();
    }
}
sout ("Second matrix");
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++) {
```

```

sout ("array [i][j] + "\t");
}
sout ();
}

```

```

int [][] sum = new int[2][3];
for (i=0; i<2; i++) {
    for (j=0; j<3; j++) {
        sum[i][j] = matrix[i][j] + array[i][j];
    }
}
sout ("sum matrix :");
for (i=0; i<2; i++) {
    for (j=0; j<3; j++) {
        sout (sum[i][j] + "\t");
    }
    sout (" ");
}

```

Q JAVA program to reverse an array

```

public class ReverseArray {
    public static void main (String[] args) {
        int[] arr = {10, 20, 30, 40, 50};
        int n = arr.length;
        for (int i=0; i<n/2; i++) {
            int temp = arr[i];
            arr[i] = arr[n-i-1];
            arr[n-i-1] = temp;
        }
        sout ("Reversed array");
        for (int i=0; i<n; i++) {
            sout (arr[i] + " ");
        }
    }
}

```

Q JAVA program to find maximum element of an array

```
int i;
int [] array = new int [5];
int largest = array [0];
sout ("enter elements :");
for (i=0; i<5; i++) {
    sout ("enter element" + (i+1) + ":");
    arr[i] = sc.next Int();
}
sout ("Array is :- ");
for (int i=0; i<5; i++) {
    sout (arr[i] + " ");
}
for (i=0; i<5; i++) {
    if (arr[i] > largest) {
        largest = arr[i];
    }
}
```

sout ("Max element is : " + largest);

Q Program to find array is sorted / not

```
int [] array = new int [5];
for (int i=0; i<5; i++) {
    sout ("enter element" + (i+1) + ":");
    array[i] = sc.next Int();
}
sout ("Array is :- ");
for (int i=0; i<5; i++) {
    sout (array[i] + " ");
}
```

```

boolean isSorted = true;
for (i = 0; i < 4; i++) {
    if (array[i] > array[i+1]) {
        isSorted = false;
        break;
    }
}

if (isSorted) {
    cout << "Sorted array";
} else {
    cout << "not sorted";
}

```

** Sorting ke lie loop humesha length minus 1 tak chalega coz last element ka $[i+1]$ nahi hai **

```

// Bubble Sort
for (i = 0; i < n-1; i++) {
    for (j = 0; j < n-i-1; j++) {
        if (array[j] > array[j+1]) {
            swap(array[j], array[j+1]);
        }
    }
}

```

→ Methods in JAVA

- Similar to functions written inside class
- Makes the task easier if we are required to do a task several times.

• Syntax :-

data type name () {

// body;

}

- This method returns sum :-

```
int mySum (int a, int b) {
```

```
    int c = a + b;
```

```
    return c; }
```

}

→ Method calling

- Method can be called by creating an object of class in which it exists, followed by method call.

`Calc obj = new Calc();` → object created

`obj.mySum(a, b);` → method called upon object

Values given to method while calling are copied to `a` & `b`. values of `mySum` function. Thus even if we modify values of `a` & `b` inside method, values in main method won't change.

→ Void return type

When we don't want our method to return anything, we use `void` as return type.

→ static keyword

It is used to associate a method of a given class with the class rather than object. static method in a class is shared by all objects.

→ Process of method invocation

Consider method sum :-

```
int sum (int a, int b)
```

```
{ return (a+b); }
```

// This method is called like this

```
calc obj = new calc ();
```

```
c = obj.sum (2,3);
```

- The values 2 & 3 are copied to a & b inside and then $a+b = 2+3 = 5$ is returned in which is an integer.

In case of arrays, reference is passed same is the case for objects passing to methods.

→ Method Overloading

Two or more methods can have same name but different parameters. Such methods are called overloaded methods.

eg :-

```
void foo ();
```

```
void foo (int a);
```

```
void foo (int a, int b);
```

} overloaded method
foo

It can't be performed by changing the return type of methods.

→ Variable Arguments (varargs)

- A function which can accommodate as many arguments we pass i.e. variable no. of arguments.

```
public static void foo (int... arr) {
// arr is available here in form of
// an array int[] array
}
```

- foo can be called with 0 or more args like foo(), foo(1, 2, 3), foo(1, 2, 3, 4, 5, 6)

- We can also create a function bar like this in which atleast one argument is passed & rest are variable

```
public static void bar (int a, int... arr) {
}
```

// code

}

and it can be called as

```
bar(1), bar(1, 2), bar(1, 2, 3, 4, 5, 6)
```

→ Recursion

- A method (function) in JAVA can call itself, such calling of function by itself is called recursion.

eg. → factorial / fibonacci

```
int fact (int n) {
    if (n == 1) return 1;
    else return n * fact(n-1);
}
```

Q JAVA method to print multiplication table of a no. in a class Table

```

void Table (int a) {
    for (int i=1; i<=10; i++) {
        cout << (a+"*"+i+"="< a*i);
    }
}

public class Main {
    public static void main {
        cout << ("Enter value of a number");
        int num = sc.nextInt();
        Table obj = new Table ();
        obj.Table(num);
    }
}

```

Q Program using functions to print

* *

* * *

* * * *

```

class Star {
    void star (int a) {
        for (int i=1; i<a; i++) {
            for (int j=0; j<=i; j++) {
                cout << "* ";
            }
            cout << "\n";
        }
    }
}

public class Main {
    public static void main (String[] args) {
        cout << ("Enter a number");
        int num = sc.nextInt();
        Star obj = new Star ();
        obj.star (num);
    }
}

```

Q Recursive function to calculate sum of first n natural numbers

```
static int sum (int n) {
    if (n == 1) {
        return 1;
    }
    else {
        return sum (n-1) + n;
    }
}
```

Q Print

```
* * * *
* * *
* *
*
```

```
class star {
    void star (int n) {
        for (int i = n; i > 0; i--) {
            for (int j = i; j > 0; j--) {
                cout << " ";
            }
            cout << "\n";
        }
    }
}
```

```
public class Main {
    public static void main (String [] args) {
        cout << "Enter no of rows : ";
        int num = sc.nextInt ();
        star obj = new star ();
        obj.star (num);
    }
}
```

Q. fibonacci series

```
class Fib {
    static int fib (int n) {
        if (n == 0 || n == 1) {
            return n;
        }
        else {
            return fib (n-1) + fib (n-2);
        }
    }
}
```

```
public class Main {
    public static void main (String[] args) {
        Scanner sc = new Scanner (System.in);
        System.out.println ("Enter a number:");
        int num = sc.nextInt();
        System.out.println (Fib.fib (num));
    }
}
```

```
public class Main {
    public static void main (String[] args) {
        Scanner sc = new Scanner (System.in);
        System.out.println ("Enter a number:");
        int num = sc.nextInt();
        System.out.println (Fib.fib (num));
    }
}
```

- Object oriented programming
- OOP tries to map code instructions with real work making the code short and easier to understand.
 - Solving a problem by creating an object is called OOP.
 - Focus on code reusability.

→ Class

A class is a blueprint for creating objects.

- Object
- An object is a real world instantiation of a class. When a class is defined, a template (info) is defined.
 - Memory is allocated only after object instantiation.

→ How to model a problem in OOP

- Identify the following:
 - Noun → Class → employee
 - Adjective → Attribute → name, age, salary
 - Verb → Methods → getSalary(), increment()

→ OOPs Terminologies

1. Abstraction

- Hides internal details & only shows essential information.
- eg → we use our phone without bothering how it was made.

2. Encapsulation

The act of putting various components together.

- eg → Laptop is single entity with wifi, speaker, storage in a single box.
- In JAVA, it simply means that sensitive data can be hidden from users.

3. Inheritance

The act of deriving new things from existing things.

- eg → rickshaw → e-rickshaw
- phone → smart phone

4. Polymorphism

- One entity can be used in many forms.
- eg → smart phone can be used as a phone as well as calculator.

→ writing a custom class

```
public class Employee {
```

```
    public int id;
```

```
    String name;
```

```
    public int getSalary () {
```

```
        // code;
```

```
    }
```

```
    public int getDetails () {
```

```
        //
```

```
    };
```

Real world obj → Properties + Behaviour

OOP's object → Attributes + Methods

→ Class

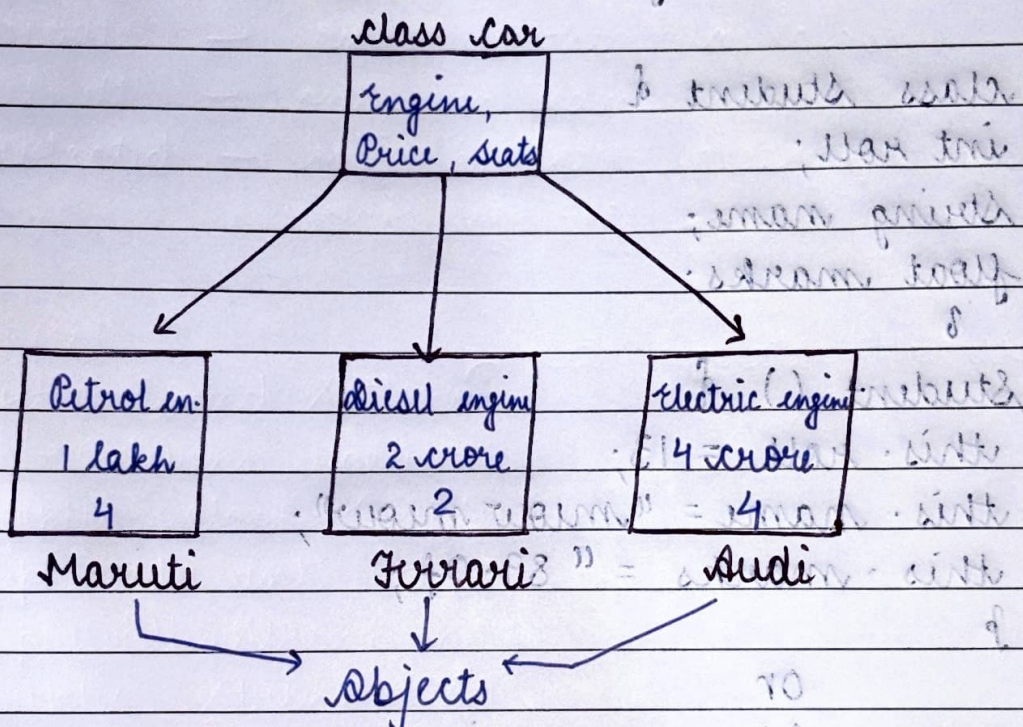
A named group of properties and functions

class student {

int roll;

float marks;

}



- class → logical construct
- object → physical reality (instance of class)

cout (Maruti . engine)

↳ gives petrol engine

→ Creation of objects

class student {

int roll; string name; float marks;

Student student1 = new Student ();

Dynamically allocates memory & returns a reference variable to it

→ Constructor

student kunal = new student(13, "Kunal", 98);

constructor

- A special function that runs when you create an object & allocates some variables

```
class Student {
    int roll;
    String name;
    float marks;
```

```
Student() {
    this.roll = 13;
    this.name = "meow meow";
    this.marks = 89.0f;
}
```

or

```
Student (int roll, String name, float marks)
{
```

```
    this.roll = roll;
    this.name = name;
    this.marks = marks;
}
```

}

(unique identifier) kuno

unique identifier kuno

unique identifier kuno

unique identifier kuno

unique identifier kuno

unique identifier kuno

unique identifier kuno

unique identifier kuno